数字电路笔记

预备知识 ⟶ 组合逻辑电路 ⟶ 时序逻辑电路

不连续、是固定量整数倍的量称为数字量，与数字量对应的信号称为数字信号，处理数字信号的电路称为数字电路。

第一章　预备知识

一、数制

1. 不同的数制

数制（符号）	基数	数码
二进制 B	2	0,1
八进制 O	8	0-7
十进制 D	10	0-9
十六进制 H	16	0-9, A-F

2. 数制转换

非十进制互转

每 3 位或 4 位分组，然后展开或缩写即可。例：

$$(11001101.011)_2 = (315.3)_8 = (CD.6)_{16}$$

非十进制转十进制

按权展开即可。例：

$$(101.01)_2 = 1\times 2^2 + 0\times 2^1 + 1\times 2^0 + 0\times 2^{-1} + 1\times 2^{-2} = (5.25)_{10}$$

$$(73.4)_8 = 7\times 8^1 + 3\times 8^0 + 4\times 8^0 + 4\times 8^{-1} = (59.5)_{10}$$

$$(3\mathrm{D.C})_{16} = 3\times 16^1 + 13\times 16^0 + 12\times 16^{-1} = (61.75)_{10}$$

十进制转非十进制

整数部分除基取余，从低到高；小数部分乘基取整，从高到低。

例：13.25

13 除以 2 余 1，6 除以 2 余 0，3 除以 2 余 1，1 除以 2 余 1，整数部分为 1101；

0.25 乘以 2 得 0.5，取整为 0，0.5 乘以 2 得 1.0，取整为 1，小数部分为 01；

故 $(13.25)_{10} = (1101.01)_2$

二、编码

1. 原码、反码与补码

编码类型	描述	缺点
原码	首位表示符号位，剩余位为二进制绝对值	减法运算困难
反码	正数与零不变；负数的符号位不变，其余位为原码取反	有两个反码表示零
补码	正数与零不变；负数的补码为反码加 1	无

补码取负

对补码取负数的方法：对补码再取一次补码。

2. 二进制编码 **二进制数码不一定表示二进制数**。

最常见的二进制编码是自然二进制码，即将其他数制的数转为二进制后的结果，不赘述。

3. 二-十进制编码（BCD 码）

编码类型	权	特性
8421 码	8, 4, 2, 1	简便直观
2421 码	2, 4, 2, 1	对 9 自补码
余 3 码	无（由 8421 码 +3 得到）	对 9 自补码
5421 码	5, 4, 2, 1	可以产生对称方波输出
Gray 码	无	相邻两数码只有一位二进制数不同

二-十六进制自然编码与 8421BCD 码区别

二-十六进制自然编码是将十进制数直接转换为十六进制数，而 8421BCD 码是将十进制数的每一位分别转换为 4 位二进制数。例：$(53)_{10} = (101011)_2 = (01010011)_{8421\text{BCD}}$

4. 可靠性编码

(1) Gray 码

Gray 码是一种特殊的二进制编码，其特点是相邻两个数码之间只有一位二进制数不同。Gray 码的主要优点是减少了在数字电路中由于信号变化引起的误差。

4 位反射 Gray 码的编码表如下：

0 (0000)	1 (0001)	2 (0011)	3 (0010)
4 (0110)	5 (0111)	6 (0101)	7 (0100)
8 (1100)	9 (1101)	10 (1111)	11 (1110)
12 (1010)	13 (1011)	14 (1001)	15 (1000)

将 9 的 Gray 码换为 1000，即得到 Gray BCD 码。

(2) 奇偶校验码

在要传输的数据中增加一个校验位，使得数据中 1 的个数为奇数或偶数，从而实现对数据传输错误的检测。

最常用的是奇校验码，因其使得全 0 成为非法码，便于检测。

由于异或门奇 1 偶 0 的特性，往往用异或门实现奇偶校验码的生成。

(3) 其他可靠性编码：Hamming 码等

三、逻辑代数

逻辑常量：0, 1

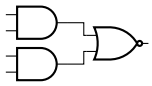
基本逻辑运算：与（AND）、或（OR）、非（NOT）

（一）逻辑运算

1. 基本逻辑运算

逻辑门	运算规则	逻辑运算式	IEEE 图形符号	IEC 图形符号
与门（AND）	有 0 则 0	$F(A, B) = A \cdot B$		
或门（OR）	有 1 则 1	$F(A, B) = A + B$		
非门（NOT）	反转输入	$F(A) = \overline{A}$		

2. 复合逻辑运算

逻辑门	运算规则	逻辑运算式	IEEE 图形符号	IEC 图形符号
与非门（NAND）	有 0 则 1	$F(A, B) = \overline{A \cdot B}$		
或非门（NOR）	有 1 则 0	$F(A, B) = \overline{A + B}$		
与或非门（AOI）	与、或、非	$F(A, B, C, D) = \overline{AB + CD}$		
异或门（XOR）	同 0 异 1	$F(A, B) = A \oplus B = \overline{A} \cdot B + A \cdot \overline{B}$		
同或门（XNOR）	同 1 异 0	$F(A, B) = A \odot B = \overline{A} \cdot \overline{B} + A \cdot B$		

（二）逻辑代数公式

1. 基本公式

名称	或式	与式
交换律	$A+B=B+A$	$A\cdot B=B\cdot A$
结合律	$(A+B)+C=A+(B+C)$	$(A\cdot B)\cdot C=A\cdot(B\cdot C)$
分配律	$A+(B\cdot C)=(A+B)\cdot(A+C)$	$A\cdot(B+C)=(A\cdot B)+(A\cdot C)$
吸收律	$A+(A\cdot B)=A$	$A\cdot(A+B)=A$
0-1 律	$A+1=1,\quad A+0=A$	$A\cdot 0=0,\quad A\cdot 1=A$
互补律	$A+\overline{A}=1$	$A\cdot\overline{A}=0$
重叠律	$A+A=A$	$A\cdot A=A$
还原律	$A+\overline{A}=1$	$A\cdot\overline{A}=0$
反演律	$\overline{A+B}=\overline{A}\cdot\overline{B}$	$\overline{A\cdot B}=\overline{A}+\overline{B}$

2. 常用公式

   - $A\cdot B+A\cdot\overline{B}=A$
   - $A+\overline{A}\cdot B=A+B$
   - $A\cdot B+\overline{A}\cdot C+B\cdot C=A\cdot B+\overline{A}\cdot C$
   - $(A+B)\cdot\left(\overline{A}+C\right)=A\cdot C+\overline{A}\cdot B$
   - $\overline{\overline{A}\cdot B+A\cdot\overline{B}}=\overline{A}\cdot\overline{B}+A\cdot B$，即 $\overline{A\oplus B}=A\odot B$

（三）逻辑代数基本规则

1. 代入规则：用逻辑表达式 F 代入某个变量 X，新逻辑表达式仍然成立。
2. 反演规则

   将原函数 F 作如下代换：

   - $+\leftrightarrow\cdot$
   - $1\leftrightarrow 0$
   - $A\leftrightarrow\overline{A}$

   即得到反函数 $\overline{F}$。反函数输出的值**与原函数相反**。
3. 对偶规则

   将原函数 F 作如下代换：

   - $+\leftrightarrow\cdot$
   - $1\leftrightarrow 0$

   即得到对偶函数 F'。对偶函数**不需要代换反变量**。

   若 F 与 G 相等，则 F' 与 G' 也相等。

（四）正负逻辑

- 正逻辑：逻辑 1 对应高电平，逻辑 0 对应低电平。
- 负逻辑：逻辑 1 对应低电平，逻辑 0 对应高电平。

正负逻辑转换

对自变量预先取反，即能将负逻辑电路转换为正逻辑电路，反之亦然。

（五）逻辑函数

1. 表示方法
   - 真值表
   - 逻辑表达式
   - 逻辑电路图
   - Karnaugh 图
   - 波形图
   - 硬件描述语言
2. 标准表达式
   - 最简与或式

     性质：所有最小项中只有一项可能是 1；$m_i \cdot m_j = 0, \quad i \neq j$；$\sum_{i=0}^{2^n-1} m_i = 1$
   - 最简或与式

     性质：所有最大项中只有一项可能是 0；$M_i + M_j = 1, \quad i \neq j$；$\prod_{i=0}^{2^n-1} M_i = 0$

（六）逻辑函数的化简

1. 代数法根据逻辑代数公式，对逻辑函数进行化简。常见方法有：
   - 并项法：$AB + A\overline{B} = A$
   - 吸收法：$A + AB = A$、$AB + \overline{A}C + BC = AB + \overline{A}C$

     还可推广为

     $$AB + \overline{A}C + BC[f(D, E, \cdots)] = AB + \overline{A}C$$
   - 消元法：$A + \overline{A}B = A + B$
2. Karnaugh 图法

Karnaugh 图的数学结构

(a) **输入空间**：对于 n 个逻辑变量，所有输入组合构成二进制空间

$$\{0,1\}^n,$$

可视为 n 维超立方体 Q_n 的 2^n 个顶点。两顶点 x, y 的 Hamming 距离为

$$d_H(x,y) = \sum_{i=1}^{n} |x_i - y_i|.$$

当 $d_H(x,y) = 1$ 时，两输入组合仅有一个变量不同，对应超立方体中的一条边。

(b) Gray **坐标**：Karnaugh 图按 Gray 码排列各坐标，使相邻码字满足

$$d_H = 1.$$

因而 Karnaugh 图中的几何邻接对应逻辑空间中的边邻接，即

$$\boxed{\text{几何相邻} \iff \text{仅一个逻辑变量发生变化}}.$$

图的上下、左右边界同样相邻，因此 Karnaugh 图具有周期边界。

(c) Descartes **积分解**：将 n 个变量分为两组，分别作为行、列坐标，则

$$Q_n \cong Q_p \square Q_q, \qquad p + q = n,$$

其中 $\square$ 表示图的笛卡尔积。例如四变量 Karnaugh 图满足

$$Q_4 \cong Q_2 \square Q_2,$$

行、列均按

$$00,\ 01,\ 11,\ 10$$

的 Gray 顺序排列，从而将四维二进制空间表示为二维坐标表。

(d) **子立方体与矩形**：一个逻辑乘积项中，未出现的变量可以自由取 0 或 1。若有 k 个自由变量，则该项对应一个含

$$2^k$$

个顶点的 k 维子立方体。

若其中 r 个自由变量属于行坐标，其余 $k-r$ 个属于列坐标，则该子立方体在 Karnaugh 图中表现为

$$2^r \times 2^{k-r}$$

的矩形，因此其面积恒为

$$\boxed{2^k}.$$

(e) **代数消元**：相邻顶点仅在一个变量上互补，因此可利用

$$AX + A\overline{X} = A(X + \overline{X}) = A$$

消去该变量。每使一个变量自由变化，相当于沿一个坐标方向扩展一维，因此

$$\boxed{2^k \text{ 个相邻格组成的子立方体} \iff \text{消去} k \text{ 个变量}}.$$

例如

$$AB\overline{C} + ABC = AB(\overline{C} + C) = AB,$$

几何上对应两个顶点沿 C 方向合并为一条边。

(f) **与或式的几何意义**：一个乘积项对应 $\{0,1\}^n$ 中一个轴对齐的子立方体；逻辑或对应这些子立方体的集合并。因此

$$F = P_1 + P_2 + \cdots + P_m$$

可理解为用若干子立方体覆盖

$$F^{-1}(1) = \{x \in \{0,1\}^n \mid F(x) = 1\}.$$

Karnaugh 图化简的本质即为：

$$\boxed{\text{用尽可能大的子立方体覆盖全部1 格}}.$$

对偶地，或与式则通过圈取 0 格，用子立方体描述 $F^{-1}(0)$。

逻辑代数	离散几何
n 个变量	n 维超立方体Q_n
一个输入组合	一个顶点
$d_H = 1$	一条边
最小项	一个顶点
一个变量自由	沿一维扩展
2^k 项合并	k 维子立方体
消去k 个变量	增加k 个自由维度
逻辑或	集合并
与或式化简	用大子立方体覆盖$F^{-1}(1)$

$$\boxed{\text{Karnaugh 图} = \text{以 Gray 码保持邻接关系的超立方体二维坐标化}}$$

第二章　门电路

（一）二极管电路

一、CMOS 门

二、TTL 门

三、ECL 门

如何计算所求电阻范围

核心约束是 $V_{OH} - (m)$

V_{OL}

第三章　组合逻辑电路

一、小规模组合电路（Small Scale Integrated Circuites, SSI）

小规模组合电路是指集成电路中逻辑门的数量不超过 10 个的组合逻辑电路。

小规模组合电路的表达式可以整理为与非 · 与非式或或非 · 或或式，从而实现为双轨输入电路。

1. 双轨输入电路

   - 与非 · 与非式

     可以运用反演律将最简与或式化为与非 · 与非式，用两级与非门实现。

     例：$F = A\overline{B} + AC = \overline{(\overline{A\overline{B}}) \cdot (\overline{AC})}$

   - 或非 · 或非式

     可以运用反演律将最简或与式化为或非 · 或非式，用两级或非门实现。

     例：$F = (A + \overline{B})(A + C) = \overline{(\overline{A + \overline{B}}) + (\overline{A + C})}$

   与非 · 与非式和或非 · 或非式各有优势，不存在绝对的优劣。判断哪种方式能最大程度简化时，应根据具体逻辑函数设计比较。

2. 单轨输入电路

   单轨输入电路可以通过给每个输入加非门实现为双轨输入电路，不多叙述。

二、冒险

（一）逻辑冒险

逻辑冒险的判断与消除方法

1. 代数式法

   只有同时存在正反变量（如 A 与 $\overline{A}$）的逻辑函数才可能出现逻辑冒险。

   将标准与或式/或与式中该正反变量以外的所有变量设为 0 或 1，将逻辑函数化为

   $$F = A + \overline{A}$$

   的形式，此时存在偏 0 型冒险；或化为

   $$F = A \cdot \overline{A}$$

   的形式，此时存在偏 1 型冒险。

   若要消除逻辑冒险，则在标准与或式/或与式中增加冗余项即可。荣誉项确定办法为将存在逻辑冒险时的所有变量组合为与式或或式，作为冗余项加入原逻辑函数中。

   例：$B = 0, C = 1$ 时存在偏 0 型冒险，冗余项为 $+\overline{B}C$；$B = 0, C = 1$ 时存在偏 1 型冒险，冗余项为 $\cdot(\overline{B} + C)$。

2. Karnaugh 图法

   判断 Karnaugh 图中是否存在相切圈，若存在则说明存在逻辑冒险现象。

   画圈使相切圈相接，即可消除逻辑冒险现象。

（二）功能冒险

在组合电路中，当有**两个或两个以上输入变量同时**发生变化，变化前后电路的输出相同，而在输入变量发生变化时可能出现瞬时错误输出，这种现象称为静态功能冒险

由状态 A 到状态 B 的过程中，若首尾电平一致，中间出现不一样的电平，则称为功能冒险。功能冒险的决定因素如下：

(1) 变量变化两个及以上；

(2) 首尾电平状态一致（从 0 到 0，从 1 到 1）；

(3) 中间电平变化（从 0 到 1，从 1 到 0）。

消除功能冒险的方法包括：

- 添加滤波电容，但会破坏输出波形；
- 加选通脉冲，对**逻辑冒险和功能冒险都有效**。

> **注意**：添加冗余项只能消除逻辑冒险而非功能冒险！

逻辑冒险与功能冒险合称静态冒险。

（三）动态冒险

在输入变化前、后函数值不同；且在输入信号变化瞬间，输出将变化三次或更高的奇数次。此处不讨论。

三、多输出函数

1. 用卡诺图分别对每个函数进行化简，并用箭头连线表示出所有的公用圈；
2. 从各个函数**相同最小项**出发，试图改变原来圈法，以求得更多的公用圈。

   原则：

   - 若改圈法后能使总圈数减少（指不同圈），则改；
   - 若使总圈数增加，则不改；
   - 若总圈数不变，则取最大的合并圈，使变量输入端减少；
   - 这一工作可重复进行多次。**注意**：**单个变量的圈不用修改**
3. 写出多输出函数的表达式，并绘出逻辑图。

四、中规模组合电路（Medium Scale Integrated Circuites, MSI）

（一）数码比较器

1. TTL 中规模集成 4 位数码比较器 74LS85

（二）编码器与优先编码器

1. 二-十位编码器 C304
2. 优先编码器 74148

（三）译码器

1. 2 线至 4 线译码器
2. 三线至八线译码器 74LS138
3. 二 - 十进制译码器（4 线-10 线译码器）

4. 七段荧光数码管显示器

（四）数据选择器

（五）数据分配器

第四章　Verilog 硬件描述语言（Verilog HDL）

Verilog HDL 既是一种行为描述语言也是一种结构描述语言，即既可以**用电路的功能描述**，也可以**用元器件及其之间的连接**来建立 Verilog HDL 模型。

Verilog HDL 的数据类型

1. 表示：位数 +'+ 数制 + 数值。例：`4'b1010` 表示 4 位二进制数 1010，`8'hFF` 表示 8 位十六进制数 FF。
2. 数制：`b` 表示二进制，`o` 表示八进制，`d` 表示十进制，`h` 表示十六进制。

`wire` 变量和 `reg` 变量

`wire` 常用于 `assign` **语句**的左值，表示连续赋值；

`reg` 常用于 `always` **块中** `=` **或** `<=` **语句**的左值。

第五章　Karnaugh 图与其数学结构